

Design Pattern

Nom du Pattern

Adam HAMOUCHE

May 3, 2026

1. Intention

On veut créer une copie integrale d'un objet sans pour autant etre dépendant de sa classe et devoir copier manuellement chacun de ses membres.

2. Structure

On implémente une interface de clonage avec une méthode clone permettant de cloner un objet. On fait heriter nos objets de l'interface prototype, chacun implemente sa propre surcharge de la fonction clone. On implemente egalement un PrototypeRegistry qui reference tous les clones recurent dans une array/hashmap.

3. Cas d'usage

Quand utiliser ce pattern ?

- Lorsqu'on veut copier un objet sans avoir accès à sa classe
- Quand on veut réduire le nombre de sous classe qui diffère que dans leur initialisation

4. Avantages

- Cloner des objets sans acceder a leur classe
- Simplifier et reduire le code d'initialisation d'objets
- Facilite la production d'objets complexes

5. Inconvénients

- Cloner des objets complexes qui ont des références circulaires peut être très délicat.

6. Exemple minimal

```
using std::string;

// Prototype Design Pattern
//
// Intent: Lets you copy existing objects without making your code
//        dependent on
//        their classes.

enum Type {
    PROTOTYPE_1 = 0,
    PROTOTYPE_2
};

/**
 * The example class that has cloning ability. We'll see how the values of
 * field
 * with different types will be cloned.
 */

class Prototype {
protected:
    string prototype_name_;
    float prototype_field_;

public:
    Prototype() {}
    Prototype(string prototype_name)
        : prototype_name_(prototype_name) {}
    virtual ~Prototype() {}
    virtual Prototype *Clone() const = 0;
    virtual void Method(float prototype_field) {
        this->prototype_field_ = prototype_field;
        std::cout << "Call Method from " << prototype_name_ << " with field: "
            << prototype_field << std::endl;
    }
};

/**
 * ConcretePrototype1 is a Sub-Class of Prototype and implement the Clone
 * Method
 * In this example all data members of Prototype Class are in the Stack.
 * If you
 * have pointers in your properties for ex: String* name_ ,you will need
 * to
 * implement the Copy-Constructor to make sure you have a deep copy from
 * the
 * clone method
 */

class ConcretePrototype1 : public Prototype {
private:
```

```

    float concrete_prototype_field1_;

public:
    ConcretePrototype1(string prototype_name, float concrete_prototype_field
        )
        : Prototype(prototype_name), concrete_prototype_field1_(
            concrete_prototype_field) {
    }

    /**
     * Notice that Clone method return a Pointer to a new ConcretePrototype1
     * replica. so, the client (who call the clone method) has the
     * responsibility
     * to free that memory. If you have smart pointer knowledge you may
     * prefer to
     * use unique_pointer here.
     */
    Prototype *Clone() const override {
        return new ConcretePrototype1(*this);
    }
};

class ConcretePrototype2 : public Prototype {
private:
    float concrete_prototype_field2_;

public:
    ConcretePrototype2(string prototype_name, float concrete_prototype_field
        )
        : Prototype(prototype_name), concrete_prototype_field2_(
            concrete_prototype_field) {
    }
    Prototype *Clone() const override {
        return new ConcretePrototype2(*this);
    }
};

/**
 * In PrototypeFactory you have two concrete prototypes, one for each
 * concrete
 * prototype class, so each time you want to create a bullet , you can use
 * the
 * existing ones and clone those.
 */

class PrototypeFactory {
private:
    std::unordered_map<Type, Prototype *, std::hash<int>> prototypes_;

public:
    PrototypeFactory() {
        prototypes_[Type::PROTOTYPE_1] = new ConcretePrototype1("PROTOTYPE_1",
            , 50.f);
    }
};

```

```

        prototypes_[Type::PROTOTYPE_2] = new ConcretePrototype2("PROTOTYPE_2",
            , 60.f);
    }

    /**
     * Be carefull of free all memory allocated. Again, if you have smart
     * pointers
     * knowelege will be better to use it here.
     */

    ~PrototypeFactory() {
        delete prototypes_[Type::PROTOTYPE_1];
        delete prototypes_[Type::PROTOTYPE_2];
    }

    /**
     * Notice here that you just need to specify the type of the prototype
     * you
     * want and the method will create from the object with this type.
     */
    Prototype *CreatePrototype(Type type) {
        return prototypes_[type]->Clone();
    }
};

void Client(PrototypeFactory &prototype_factory) {
    std::cout << "Let's create a Prototype 1\n";

    Prototype *prototype = prototype_factory.CreatePrototype(Type::
        PROTOTYPE_1);
    prototype->Method(90);
    delete prototype;

    std::cout << "\n";

    std::cout << "Let's create a Prototype 2\n";

    prototype = prototype_factory.CreatePrototype(Type::PROTOTYPE_2);
    prototype->Method(10);

    delete prototype;
}

int main() {
    PrototypeFactory *prototype_factory = new PrototypeFactory();
    Client(*prototype_factory);
    delete prototype_factory;

    return 0;
}

```

7. Exemple concret (jeu / moteur)

Exemple réel :

- Systeme de prefab
- ECS -i Prototype de composants
- Systeme d'ennemis

8. Sources

<https://refactoring.guru/design-patterns/prototype>